

Factory Method Design Pattern in Java

If you are a novice or experienced java programmer, design pattern is something you should have in your arsenal. This is important if you are going to attend an interview or developing an enterprise level application. One of the most important thing about design pattern is apply the right pattern at right place.

Let us examine a practical scenario which is suitable for factory method pattern.

We are developing an enterprise application. You are responsible for authentication module. But you are not sure how you are going to do authentication. You can have authentication against LDAP, database or any other centralized directory. So you first define an interface for authentication with single method authorize() This method returns true if user is successfully authenticated or false if not authenticated.

```
public interface UserAuthentication {  
  
    public boolean authorize();  
  
}
```

OK, now let us create concrete implementation for this interface. Since you can have multiple ways of authenticating against users, you would like to create multiple derived classes.

Let us create a class which authenticate against users table in database using JDBC.

```
public class JDBCUserAuthentication implements UserAuthentication {  
  
    @Override  
    public boolean authorize() {  
        boolean result=false;  
        // implementation for authentication using JDBC  
        //if authenticated  
        //result=true  
        //else  
        //result=false  
        return result;  
    }  
  
}
```

Similarly another class which authenticate against LDAP.

```
public class LDAPUserAuthentication implements UserAuthentication {  
  
    @Override  
    public boolean authorize() {  
        boolean result=false;  
        // implementation for authentication using LDAP  
        //if authenticated  
        //result=true  
        //else  
        //result=false  
        return result;  
    }  
  
}
```

Well, now we have two implementations, one using JDBC and another using LDAP. Now let us create factory class.

```
public class UserAuthenticationFactory {  
  
    public static UserAuthentication getUserAuthentication(String type){  
  
        if (type.equalsIgnoreCase("jdbc")){  
            return new JDBCUserAuthentication();  
        }else if(type.equalsIgnoreCase("ldap")){  
            return new LDAPUserAuthentication();  
        }else{  
            return new JDBCUserAuthentication();  
        }  
    }  
  
}
```

In the above class we define a static method getUserAuthentication() which instantiates appropriate sub class based on parameter.

When you do authentication your code snippet may be as follows.

```
//declare isAuthorized variable
```

```
boolean isAuthorized=false;
//get UserAuthentication object
UserAuthentication userAuthentication=
    UserAuthenticationFactory.getUserAuthentication("jdbc");

isAuthorized=userAuthentication.authorize();
if(isAuthorized){
    //redirect to home page
}else{
    //redirect to login page
}
```

Here we call static method of our factory class which in turn instantiates JDBCUserAuthentication class because we pass parameter as as “jdbc”. But the user who write client call for authentication do not require to know which derived class is instantiated and its implementation details.

I hope you got enough clarity about factory method pattern and its implementation in practical scenario.

cheers,
